index

Symbols

@AuthenticationPrincipal annotation 388, 441

@Autowired annotation 117

@Component annotation 115

@ComponentScan annotation 48

@ConditionalOnCloudPlatform annotation 124

@ConditionalOnProperty annotation 124

@Configuration annotation 48

@Configuration class 115 – 116

@ConfigurationProperties annotation 118 – 119

@ConfigurationProperties bean 145, 273

@ConfigurationPropertiesScan annotation 118, 274

@CreatedBy annotation 163

@CreatedDate annotation 163, 265

@CurrentSecurityContext annotation 388

@DataJdbcTest annotation 170 – 172

@DataR2dbcTest annotation 286, 289 – 290

@DynamicProperties annotation 289

@EnableAutoConfiguration annotation 49

@EnableConfigServer annotation 135

@EnableConfigurationProperties annotation 118

@EnableJdbcAuditing annotation 163, 433

@EnableJpaAuditing annotation 163

@EnableR2dbcAuditing annotation 267

@Entity annotation 160

@EventListener(ApplicationReadyEvent.class) annotation 122

@FunctionalSpringBootTest annotation 342 – 343

@Id annotation 158, 265

@JsonTest annotation 101 – 103

@LastModifiedBy annotation 164

@LastModifiedDate annotation 164, 265

@MockBean annotation 101

@MockBean Spring annotation 291

@NativeHints annotation 558

@PreAuthorize annotation 433

@Profile annotation 122

@PropertySource annotations 116

@Query annotation 167

@RefreshScope annotation 145

@RestControllerAdvice bean 92 – 93, 100

@RestController annotation 50

@RestController bean 100

@RestController class 87, 271, 308, 366

@Service class 84

@SpringBootApplication annotation 48

@SpringBootTest annotation 50, 97 – 100, 170, 172 – 173, 292

@SpringWebFlux annotation 400

@Table annotation 265

@Testcontainers annotation 289

@TestPropertySource annotation 116, 319

@Transactional annotation 168

@TypedHint annotation 558

@Valid annotation 91, 271

@Value annotation 117 – 118, 127

@Version annotation 159

@WebFluxTest annotation 286

testing REST controllers with 290 – 292

testing secured REST controllers with 432 – 433

@WebMvcTest annotation 100

testing REST controllers with 100 – 101

testing secured REST controllers with 423 – 425

@WithMockUser annotation 436, 441

testing data auditing and protection with 441 – 443

testing data auditing with 436 – 437

/actuator/beans endpoint 479

/actuator/configprops endpoint 479

/actuator/env endpoint 479

/actuator/flyway endpoint 479

/actuator/health endpoint 456, 479

/actuator/health/liveness state 460

/actuator/health/readiness state 460

/actuator/heapdump endpoint 479

/actuator/info endpoint 479

/actuator/liquibase endpoint 480

/actuator/loggers enpoint 479

/actuator/mappings endpoint 479

/actuator/metrics endpoint 465, 479

/actuator/prometheus endpoint 479

/actuator/refresh endpoint 145 – 146, 488, 499

/actuator/sessions endpoint 479

/actuator/threaddump endpoint 479

/books endpoint 83, 87, 369, 478

/books/{bookIsbn} endpoint 275 – 276

/books/{isbn} endpoint 83

/catalog-service/prod endpoint 139

./create-cluster.sh script 404

/decrypt dedicated endpoint 490

/encrypt dedicated endpoint 490

./gradlew bootRun application 89, 93, 175, 398 – 399, 458, 461, 465 – 466

/logout endpoint 390, 398, 402

/monitor endpoint 489

/oauth2/authorization/keycloak login endpoint 396

/oauth2/authorization/{registrationId} endpoint 396

/orders endpoint 261, 310, 440

/user endpoint 386 – 389, 401, 420

Numerics

12-Factor methodology. *See* 15-Factor methodology

15-Factor methodology 36 – 42

administrative processes 40

API first 37

authentication and authorization 41 – 42

backing services 39

concurrency 41

configuration, credentials, and code 38

dependency management 37

design, build, release, run stages 38

disposability 39

environment parity 39 – 40

logs 38 – 39

one codebase, one application 37

port binding 40

stateless processes 41

telemetry 41

2FA (two-factor authentication) 376

A

Accept:text/event-stream header 570

acceptance stage, in deployment pipeline 103

Accept header 90, 466

Access Tokens

extracting user roles from 421 – 422

storing in Redis 411 – 412

ACK (Alibaba Container Service for Kubernetes) 9

ACM Queue (Barr) 20

administrative processes 40, 144

Advanced Message Queuing Protocol (AMQP) 331 – 332

Agile Testing Condensed (Crispin and Gregory) 95

Ahead-Of-Time (AOT) compilation 557

AKS (Azure Kubernetes Service) 9

allQuotes function 570

Amazon EKS (Elastic Kubernetes Service) 9

Amazon RDS (Relational Database Service) 151

AMQP (Advanced Message Queuing Protocol) 331

AND conditionals 301

AND logical operator 301

andThen() operator 340

Angular application 393 – 395

annotationProcessor scope 274

AOT (Ahead-Of-Time) compilation 557

API gateway

distributed session management with Redis 316 – 320

edge servers and Spring Cloud Gateway 295 – 304

bootstrapping edge server with Spring Cloud Gateway 297 – 299

defining routes and predicates 299 – 303

processing requests and responses through filters 303 – 304

fault tolerance with Spring Cloud Circuit Breaker and Resilience4J 304 – 311

circuit breakers with Spring Cloud Circuit Breaker 306

combining circuit breakers, retries, and time limiters 309 – 311

configuring circuit breaker with Resilience4J 307

defining fallback REST APIs with Spring WebFlux 308

managing external access with Kubernetes Ingress 320 – 326

Ingress API and Ingress Controller 320 – 323

working with Ingress objects 323 – 326

request rate limiting with Spring Cloud Gateway and Redis 311 – 316

configuring request rate limiter 314 – 316

integrating Spring with Redis 313 – 314

running Redis as container 312 – 313

APIs

15-Factor methodology 37

evolving for future requirements 94

protecting 417 – 433

role-based access control with Spring Security and JWT 420 – 423

securing Spring Boot as OAuth2 Resource Server 417 – 420, 429 – 431

testing OAuth2 with Spring Security and Testcontainers 423 – 428, 431 – 433

API versioning 94

apiVersion section, of Kubernates manifest 223 – 224

app_delivery event 544 – 545

{application} parameter 132, 138, 141

application layer 194

application properties 117 – 118

application server 76

application services 32

architectures 29 – 33

asynchronous and non-blocking 255 – 259

from thread-per-request to event loop 255 – 257

Project Reactor 257 – 258

Spring reactive stack 258 – 259

event-driven applications 328 – 331

event-driven models 329

using pub/sub model 329 – 331

multi-tiered and microservices architectures 30 – 32

service-based architecture 32 – 33

application services 32

data services 33

interactions 33

ARG instruction 183

Argo CD 549 – 552

argocd app get catalog-service command 552

Artifact parameter 45

asynchronous architectures 255 – 259

from thread-per-request to event loop 255 – 257

Project Reactor 257 – 258

Spring reactive stack 258 – 259

atomicity 358 – 363

auditing data

enabling and configuring JDBC 163 – 165

protecting user data with Spring Security and Spring Data R2DBC 437 – 441

adding audit metadata for users who created or updated R2DBC data entity 438 – 439

defining auditor to capture who created or updated R2DBC data entity 437 – 438

ensuring user data privacy 440 – 441

writing Flyway migration to add new audit metadata to schema 439

testing data auditing and protection with @WithMockUser and Spring Data R2DBC 441 – 443

testing data auditing with Spring Data and @WithMockUser 436 – 437

with Spring Security and Spring Data JDBC 433 – 436

adding audit metadata for users who created or updated JDBC data entity 434 – 435

defining auditor to capture who created or updated JDBC data entity 433 – 434

writing Flyway migration to add new audit metadata to schema 435 – 436

AuditorAware bean 433 – 434

authentication 374 – 382

15-Factor methodology 41 – 42

authenticating users with OpenID Connect 374 – 378

exchanging user information with JWT 378 – 379

registering application in Keycloak 379 – 382

with Spring Security and OpenID Connect 382 – 393

adding new dependencies 383

basic Spring Security configuration 384 – 385

configuring integration between Spring Security and Keycloak 383 – 384

configuring user logout in Spring Security and Keycloak 390 – 393

inspecting authenticated user context 386 – 390

See also *security*

Authentication object 387, 420, 433

authorization

15-Factor methodology 41 – 42

with Spring Cloud Gateway and OAuth2 407 – 417

customizing tokens and propagating user roles 412 – 417

token relay from Spring Cloud Gateway to other services 409 – 412

Authorization Code 376

Authorization Code flow, OAuth2 376

Authorization header 410 – 411, 417 – 418, 427

Authorization Server 374 – 375, 407 – 408

automation 16 – 17

autoscaling.knative.dev/minScale annotation 579

az postgres server create command 149

Azure Kubernetes Service (AKS) 9

B

BaaS (Backend as a Service) model 28

backend-for-frontend pattern 295

backing services 39, 228

Barr, Jeff 20

basedOnPreviousValue parameter 303

bases 506 – 507

{baseUrl} placeholder 391

bearer tokens 412

Beyond the Twelve-Factor App (Hoffman) 36

binding functions 346 – 351

configuring destination bindings 347 – 348

destination bindings 347

exploring exchanges and queues in RabbitMQ 349 – 351

binding term 346

blocking 78

blue/green deployments 540

blue production environment 540

Blyler, J. E. 13

bootBuildImage Gradle task 55, 565

bootJar Gradle task 77

bootRun Gradle task 50 – 51, 77, 123

bootstrapping

building Spring Boot application 48 – 49

cloud native project 72 – 75

dependency management with Gradle and Maven 73 – 75

one codebase, one application 72 – 73

configuration server 133 – 135

edge server with Spring Cloud Gateway 297 – 299

project with Spring Native and Spring Cloud Function 567 – 568

project with Spring Reactive Web and Spring Native 558 – 559

reactive application with Spring Boot 261 – 263

BPL_JVM_THREAD_COUNT environment variable 204, 537

BP_NATIVE_IMAGE environment variable 565

brew install kustomize command 532

Brown, Simon 12

build artifact 38

buildDispatchedOrder() method 439

builder image 198

build Gradle task 78

Building Microservices (Newman) 30, 355

buildpacks 198

BuildProperties object 481

builds 72

build stage

15-Factor methodology 38

configuring Spring Boot application 47 – 48

Polar Bookshop 67

bulkhead pattern 312

business key 159

C

CaaS (Container as a Service) model 9, 26

calendar versioning 520

canary release 540

Carnell, John 30

catalog-config ConfigMap 495, 503 – 504, 511 – 512, 531

CatalogServiceApplication class 48, 118

CatalogServiceApplicationTests class 50, 98, 172, 425

catalog-service container 455, 495, 513, 532 – 533

catalog-service DNS name 234

catalog-service entry 201

catalog-service GitHub repository 211, 528, 532

catalog-service Git repository 44

catalog-service package 201

catalog-service repository 248, 509, 512, 528, 544 – 545

CCPA (California Consumer Privacy Act) 7, 151

CD (continuous delivery) 17 – 19, 547 – 554

acceptance stage 519 – 528

acceptance stage of 523 – 524

implementing acceptance stage with GitHub Actions 525 – 528

versioning release candidates for continuous delivery 519 – 522

configuring Spring Boot for production 528 – 540

configuring CPU and memory for Spring Boot containers 534 – 538

defining configuration overlay for production 528 – 534

deploying Spring Boot in production 539 – 540

continuous deployment with GitOps 547 – 554

production stage 540 – 547

implementing with GitHub Actions 541 – 547

overview 541

centralized configuration management 128 – 139

configuration server REST API 138 – 139

making configuration server resilient 137 – 138

setting up configuration server 133 – 137

bootstrapping project 133 – 135

configuring configuration server 136 – 137

enabling configuration server 135 – 136

using Git to store configuration data 131 – 133

cgroups 25

chaos engineering 524

charts 220, 597

Chester, Jaques 580

chmod +x create-cluster.sh command 323

chmod +x deploy.sh command 549, 593, 597 – 599

chmod +x destroy-cluster.sh command 326

chmod +x install-knative.sh command 575

CI (Continuous integration) 18

CircuitBreaker filter 305 – 309

circuit breakers

combining retries and time limiters 309 – 311

configuring with Resilience4J 307

with Spring Cloud Circuit Breaker 306

claims 378, 412

Claims Set objecct 378

Client 374 – 375, 407 – 408

ClientConfig class 276

ClientProperties record 273

clients 332

client-side rate limiters 311

client-side service discovery 229 – 231

closed state 304 – 305

cloud

deleting resources 600

deploying serverless applications on 573 – 574

overview 6 – 10

CloudEvents 571

cloud native

architectural elements for 29 – 33

multi-tiered and microservices architectures 30 – 32

service-based architecture 32 – 33

cloud and cloud computing model 6 – 10

CaaS (Container as a Service) 9

FaaS (Function as a Service) 9

IaaS (Infrastructure as a Service) 8

PaaS (Platform as a Service) 9

SaaS (Software as a Service) 9 – 10

culture and practices supporting 15 – 20

automation 16 – 17

CD (continuous delivery) 17 – 19

DevOps 19 – 20

databases for cloud native systems 149 – 154

data services in cloud 149 – 152

running PostgreSQL as container 153 – 154

defined 4 – 6

properties of cloud native applications 10 – 15

loose coupling 12 – 13

manageability 15

observability 14 – 15

resilience 13

scalability 11 – 12

reasons for choosing 21 – 23

cost 23

resilience 22 – 23

scale 23

speed 21 – 22

topologies 24 – 29

containers 24 – 26

orchestration 27 – 28

serverless technologies 28 – 29

cloud native applications

15-Factor methodology 36 – 42

administrative processes 40

API first 37

authentication and authorization 41 – 42

backing services 39

concurrency 41

configuration, credentials, and code 38

dependency management 37

design, build, release, run stages 38

disposability 39

environment parity 39 – 40

logs 38 – 39

one codebase, one application 37

port binding 40

stateless processes 41

telemetry 41

building with Spring 42 – 51

building Spring Boot application 43 – 51

Spring landscape 42 – 43

containerizing applications with Docker 51 – 56

images and containers 52 – 54

running Spring application as container 55 – 56

management and monitoring 479 – 484

exposing application information 481 – 482

Flyway migrations in Spring Boot 480 – 481

generating and analyzing heap dumps 483 – 484

managing containers with Kubernetes 57 – 62

deployments, pods, and services 58 – 59

running Spring application on Kubernetes 59 – 62

Polar Bookshop 62 – 67

exploring patterns and technologies used in project 63 – 67

requirements of system 62 – 63

Cloud Native Buildpacks 26

building container images 197

containerizing native images with 564 – 565

containerizing Spring Boot with 197 – 201

cloud native development

bootstrapping cloud native project 72 – 75

dependency management with Gradle and Maven 73 – 75

one codebase, one application 72 – 73

building RESTful application with Spring MVC 82 – 94

data validation and error handling 90 – 93

evolving APIs for future requirements 94

implementing REST API with Spring MVC 87 – 90

REST API first, business logic later 82 – 87

deployment pipeline 103 – 110

commit stage of 104 – 105

implementing commit stage with GitHub Actions 105 – 110

testing RESTful application with Spring 94 – 103

integration tests with @SpringBootTest 97 – 99

testing JSON serialization with @JsonTest 101 – 103

testing REST controllers with @WebMvcTest 100 – 101

unit tests with JUnit 5 96 – 97

working with embedded servers 75 – 81

configuring embedded Tomcat 80 – 81

executable JARs and embedded servers 76 – 78

thread-per-request model 78 – 80

Cloud Native (Fremantle) 4

Cloud Native Patterns (Davis) 32

cloud platform services 129

cloud vendor services 129

ClusterIP type 233, 235, 320

clusters 58, 217

CMD instruction 183

CNCF (Cloud Native Computing Foundation) 5

code, in 15-Factor methodology 38

codebases

15-Factor methodology 37

bootstrapping cloud native project 72 – 73

code linting 104

cohesion 12

command-line arguments 126

commit stage

building release candidates in 207 – 208

implementing with GitHub Actions 105 – 110

overview 104 – 105

validating Kubernetes manifests in 246 – 247

commit stage, in deployment pipeline 103

compliance property 151

com.polarbookshop.catalogservice.config package 118, 163, 419

com.polarbookshop.catalogservice.demo package 122

com.polarbookshop.catalogservice.domain package 84 – 85

com.polarbookshop.catalogservice.persistence package 86

com.polarbookshop.catalogservice.web package 92

com.polarbookshop.dispatcherservice package 337, 339

com.polarbookshop.edgeservice.config package 315, 367

com.polarbookshop.edgeservice.user package 386 – 387

com.polarbookshop.edgeservice.web package 308

com.polarbookshop.orderservice.book package 276 – 277

com.polarbookshop.orderservice.config package 267, 273, 276, 430

com.polarbookshop.orderservice.order.domain package 265 – 267, 269

com.polarbookshop.orderservice.order.event package 356, 358

com.polarbookshop.orderservice.order.web package 271

com.polarbookshop.quotefunction.domain package 568

com.polarbookshop.quotefunction.functions package 568

com.polarbookshop.quoteservice.domain package 559, 568

com.polarbookshop.quoteservice.web package 561

compose() operator 340

concurrency, 15-Factor methodology 41

concurrent request limiter 312

confidential Client 379 – 380

ConfigMaps 491 – 501

configuring Spring Boot with 492 – 497

customizing 511 – 512, 531

refreshing configuration at runtime with Spring Cloud Kubernetes 499 – 501

storing sensitive information with Secrets 497 – 499

config-repo repository 131, 141, 145

ConfigServiceApplication class 135

config-service repository 134

config trees 494

configuration

15-Factor methodology 38

configuring applications on Kubernetes 487 – 491

disabling Spring Cloud Config 491

managing secrets with Spring Cloud Config 490 – 491

refreshing configuration at runtime with Spring Cloud Bus 488 – 490

securing configuration server with Spring Security 488

management with Kustomize 501 – 516

customizing ConfigMaps 511 – 512

customizing environment variables 510 – 511

customizing image name and version 512 – 513

customizing number of replicas 514 – 516

defining configuration overlay for staging 509 – 510

managing Kubernetes configuration for multiple environments with 506 – 509

using to manage and configure Spring Boot applications 502 – 506

Polar Bookshop 65 – 66

use of term 115

using ConfigMaps and Secrets in Kubernetes 491 – 501

configuring Spring Boot with ConfigMaps 492 – 497

refreshing configuration at runtime with Spring Cloud Kubernetes 499 – 501

storing sensitive information with Secrets 497 – 499

configuration as code 16

configuration groups 124

configuration server

making resilient 137 – 138

REST API 138 – 139

setting up 133 – 137

bootstrapping project 133 – 135

configuring configuration server 136 – 137

enabling configuration server 135 – 136

using with Spring Cloud Config Client 140 – 146

making configuration client resilient 142 – 143

refreshing configuration at runtime 143 – 146

setting up configuration client 140 – 142

configuration services 114, 129, 487

ConnectException type 311

connection factories 154

connection pooling 157

connection pool timeout 280

connection timeout 80 – 81, 280

constructor-based dependency injection 85

consumeOrderDispatchedMessageEvent() method 357

Consumer function 335, 339, 567

consumer groups 348

Consumer objects 356

Container as a Service (CaaS) model 9, 26

container images 53

deployment pipeline 207 – 213

building release candidates in commit stage 207 – 208

publishing container images with GitHub Actions 208 – 213

managing Spring Boot containers with Docker Compose 202 – 207

debugging Spring Boot containers 204 – 207

using Docker Compose to manage container life cycle 203 – 204

native images with Buildpacks 564 – 565

packaging Spring Boot applications as 188 – 201

building container images for production 193 – 197

containerizing Spring Boot with Cloud Native Buildpacks 197 – 201

containerizing Spring Boot with Dockerfiles 191 – 193

preparing Spring Boot for containerization 189 – 191

working with on Docker 180 – 188

container images, overview of 181 – 182

creating images with Dockerfiles 182 – 185

publishing images on GitHub Container Registry 185 – 188

container layer 181

container registry 52, 187

containers 26, 44

assigning resource requests and limits to 535 – 536

containerizing applications with Docker 51 – 56

images and containers 52 – 54

running Spring application as container 55 – 56

managing with Kubernetes 57 – 62

deployments, pods, and services 58 – 59

running Spring application on Kubernetes 59 – 62

overview 24 – 26

running PostgreSQL as 153 – 154

running Redis as 312 – 313

content negotiation 90

continuous delivery 541

Continuous Delivery for Kubernetes (Salatino) 580, 590

Continuous Delivery (Humble and Farley) 18 – 19, 95, 103

continuous delivery pipelines 18

Continuous Delivery Pipelines (Farley) 19, 103, 521

continuous delivery. *See* CD (continuous delivery)

continuous deployment 18, 541

continuously reconciled principle 548

control flow 257

controllers

implementing for Spring Boot application 49 – 50

testing REST controllers 100 – 101

control plane 58, 217

CookieServerCsrfTokenRepository class 399

COPY instruction 183

copy-on-write 181

CORS (Cross-Origin Request Sharing) 393, 395

CORS in Action (Hossain) 396

cost 23

coupling 12

CPU 534 – 538

assigning resource requests and limits to container 535 – 536

configuring resources for JVM 537 – 538

optimizing for Spring Boot applications 536 – 537

crack propagation 280

CRDs (Custom Resource Definitions) 577

credentials, 15-Factor methodology 38

Crispin, Lisa 95

Cross-Origin Request Sharing (CORS) 393, 395

CrudRepository interface 167

CSRF (Cross-Site Request Forgery)

protecting against 398 – 400

testing 402 – 404

curl command 46, 89, 134, 261, 297, 337

customer role 373, 414, 417

customers user 373

Customizer bean 307

Customizer interface, Spring Security 368

custom properties

defining 118 – 120

using 120 – 121

Custom Resource Definitions (CRDs) 577

D

data 433 – 443

auditing with Spring Security and Spring Data JDBC 433 – 436

adding audit metadata for users who created or updated JDBC data entity 434 – 435

defining auditor to capture who created or updated JDBC data entity 433 – 434

writing Flyway migration to add new audit metadata to schema 435 – 436

Polar Bookshop 65

protecting user data with Spring Security and Spring Data R2DBC 437 – 441

adding audit metadata for users who created or updated R2DBC data entity 438 – 439

defining auditor to capture who created or updated R2DBC data entity 437 – 438

ensuring user data privacy 440 – 441

writing Flyway migration to add new audit metadata to schema 439

testing data auditing and protection with @WithMockUser and Spring Data R2DBC 441 – 443

testing data auditing with Spring Data and @WithMockUser 436 – 437

using repository abstraction for access 85 – 86

database auditing 163

database drivers 154

database-per-service 264

databases

data persistence with Spring Data JDBC 154 – 169

connecting to database with JDBC 156 – 158

data repositories with Spring Data 166 – 169

defining persistent entities with Spring Data 158 – 163

enabling and configuring JDBC auditing 163 – 165

for cloud native systems 149 – 154

data services in cloud 149 – 152

running PostgreSQL as container 153 – 154

managing database schemas with Flyway 267 – 269

managing databases in production with Flyway 173 – 177

evolving database with Flyway 176 – 177

initializing database schema with Flyway 175 – 176

version control for database 173 – 175

testing data persistence with Spring and Testcontainers 169 – 173

configuring Testcontainers for PostgreSQL 170 – 171

integration tests with @SpringBootTest and Testcontainers 172 – 173

testing data persistence with @DataJdbcTest and Testcontainers 171 – 172

DataConfig class 163, 267, 433, 438

data persistence

persisting data reactively with Spring Data R2DBC 263 – 269

connecting to database with R2DBC 265

defining persistent entities 265 – 267

managing database schemas with Flyway 267 – 269

running PostgreSQL database for Order Service 264

using reactive repositories 267

testing with @DataR2dbcTest and Testcontainers 289 – 290

testing with Spring and Testcontainers 169 – 173

configuring Testcontainers for PostgreSQL 170 – 171

integration tests with @SpringBootTest and Testcontainers 172 – 173

testing data persistence with @DataJdbcTest and Testcontainers 171 – 172

with Spring Data JDBC 154 – 169

connecting to database with JDBC 156 – 158

data repositories with Spring Data 166 – 169

defining persistent entities with Spring Data 158 – 163

enabling and configuring JDBC auditing 163 – 165

data repositories 166 – 169

defining transactional contexts 168 – 169

using 166 – 168

data services 33, 149 – 152

categories of 151 – 152

challenges with 150 – 151

managing in local cluster 220 – 221

data streams 340 – 342

data validation 90 – 93

Davis, Cornelia 10, 32

DDD (domain-driven design) 156

debugging Spring Boot containers 204 – 207

declarative configuration 223

declarative principle 547

defaultIfEmpty() operator 278

DEFINED_PORT Web environment option 98

degraded functionality 279

DELETE request 398, 400, 425, 428

deleting cloud resources 600

denial-of-service (DoS) attack 80

deny rule 547

dependencies layer 194

Dependencies parameter 46

dependencies section, build.gradle file 262, 298, 337, 559

dependency management 72

15-Factor methodology 37

adding new dependencies 383

with Gradle and Maven 73 – 75

deployment coupling 328

deployment pipeline 18

acceptance stage 519 – 528

implementing with GitHub Actions 525 – 528

overview 523 – 524

versioning release candidates for continuous delivery 519 – 522

build and publish serverless applications 571 – 573

cloud native development 103 – 110

commit stage of 104 – 105

implementing commit stage with GitHub Actions 105 – 110

container images 207 – 213

building release candidates in commit stage 207 – 208

publishing container images with GitHub Actions 208 – 213

Kubernetes 58 – 59, 245 – 248

automating Kubernetes manifests validation with GitHub Actions 247 – 248

validating Kubernetes manifests in commit stage 246 – 247

Polar Bookshop 67

production stage 540 – 547

implementing with GitHub Actions 541 – 547

overview 541

Deployment resource 60

Deployments for Spring Boot 221 – 228

controlling Pods with Deployments 222 – 223

creating Deployment for Spring Boot application 223 – 228

creating Deployment object from manifest 227 – 228

defining Deployment manifest with YAML 224 – **227**

from containers to Pods 221

design stage, 15-Factor methodology 38

destination bindings 344, **347**

configuring 347 – **348**

overview 347

DevOps 19 – **20**

dev profile 121

DigitalOcean

running Kubernetes cluster on 590 – **593**

running PostgreSQL database on 593 – **595**

running Redis on 595 – **596**

DigitalOcean Kubernetes (DOKS) 591

disableNameSuffixHash: true generator option 505

dispatcher-service Git repository 336

DispatcherServlet component 79, **87**

dispatch function 347

DispatchingFunctions class 338 – **340**

DispatchingFunctionsIntegrationTests class 342, **352**

dispatchOrder function 357 – **358**

dispatchOrder-in-0 binding 358

disposability

15-Factor methodology 39

Kubernetes 236 – 240

fast startup 236

graceful shutdown 236 – 239

scaling Spring Boot applications 239 – 240

distributed monolith 12, 72

distributed session management 316 – 320

distributed tracing 472 – 479

configuring tracing in Spring Boot with OpenTelemetry 475 – 479

managing traces with Tempo and Grafana 474 – 475

DNS server 189 – 191

Docker

containerizing applications with 51 – 56

images and containers 52 – 54

running Spring application as container 55 – 56

moving to Kubernetes from 216 – 221

managing data services in local cluster 220 – 221

working with local Kubernetes cluster 218 – 219

setting up 583 – 585

using built-in DNS server for service discovery 189 – 191

working with container images on 180 – 188

container images, overview of 181 – 182

creating images with Dockerfiles 182 – 185

publishing images on GitHub Container Registry 185 – 188

docker build command 184, 192

Docker CE (Docker Community Edition) 584

Docker client 52

Docker Compose 202 – 207

debugging Spring Boot containers 204 – 207

using to manage container life cycle 203 – 204

docker-compose 363, 423, 431, 461, 466, 470, 479, 484

docker-compose—version command 203

Docker container 26

Docker daemon 52

Docker Desktop for Mac 54

Docker Desktop for Windows 54

Dockerfiles

building container images 197

containerizing Spring Boot with 191 – 193

creating images with 182 – 185

Docker host 52

Docker image 53, 179

docker images command 184, 585

Docker in Practice (Meill and Sayers) 207

docker.io/library/postgres qualified name 187

docker.io/library/ubuntu qualified name 187

docker.io hostname 187

docker logs catalog-service container 477

docker logs <name> Docker CLI command 585

docker ps -a command 585

docker ps command 585

docker rm -fv polar-postgres database container 169

docker rm -fv polar-postgres PostgreSQL container 176

docker rm <image> command 585

docker rm <name> command 585

docker run command 154, 185, 192

Docker server 52

docker start <name> command 585

docker stop <name> command 585

docker tag command 187

docker version command 54, 180

doctl command 594, 596

doctl compute size list command 591

DOKS (DigitalOcean Kubernetes) 591

domain class, making persistent 158 – 161

domain-driven design 87

domain-driven design (DDD) 156

domain entity 84, 265

DoS (denial-of-service) attack 80

E

edge servers 295 – 304

bootstrapping with Spring Cloud Gateway 297 – 299

defining routes and predicates 299 – 303

processing requests and responses through filters 303 – 304

EdgeServiceApplicationTests class 319

edge-service client identifier 384

edge-service Git repository 297

Edge Service project 306

EKS (Elastic Kubernetes Service) 9

elastic connection pool 302

Elastic Kubernetes Service (Amazon EKS) 9

embedded property file 494

embedded servers 75 – 81

configuring embedded Tomcat 80 – 81

connection timeout 80 – 81

HTTP port 80

thread pool 81

executable JARs and embedded servers 76 – 78

thread-per-request model 78 – 80

entities 83, 155

ENTRYPOINT instruction 183

Environment abstraction 115 – 116

Environment interface 117, 127

Environment object 117, 126

environment parity, 15-Factor methodology 39 – 40

environments

Docker 583 – 585

Java 582 – 583

Kubernetes 585 – 587

tools 587 – 588

grype 587

HTTPie 587

Knative CLI 588

Kubeval 588

Octant 588

Tilt 588

environment variables 114, 487

configuring application through 127 – 128

customizing 510 – 511, 529

ErrImagePull error status 227

ERROR event 310

error handling 285 – 286

building RESTful application with Spring MVC 90 – 93

using exceptions to signal errors in domain 87

event consumers 328, 355 – 358

event-driven applications

event-driven architectures 328 – 331

event-driven models 329

using pub/sub model 329 – 331

functions with Spring Cloud Function 335 – 343

composing and integrating functions 340 – 342

using functional paradigm in 335 – 340

writing integration tests with @FunctionalSpringBootTest 342 – 343

message brokers with RabbitMQ 331 – 335

AMQP for messaging systems 332

using RabbitMQ for publish/subscribe communications 333 – 335

processing messages with Spring Cloud Stream 344 – 355

binding functions to message channels 346 – 351

configuring integration with RabbitMQ 344 – 346

making messaging resilient to failures 354 – 355

writing integration tests with test binder 352 – 354

producing and consuming messages with Spring Cloud Stream 355 – 363

implementing event consumers, and problem of idempotency 355 – 358

implementing event producers, and problem of atomicity 358 – 363

event logs 448

event loop 255 – 257

event producers 328, 358 – 363

Event streaming model 329

eventual consistency 151

exceptions 87

exchanges 332, 367, 389

data 275 – 276

in RabbitMQ 349 – 351

executable specifications 523

exploratory testing 524

exponential backoff strategy 283

exposing application information 481 – 482

external access 320 – 326

Ingress API and Ingress Controller 320 – 323

working with Ingress objects 323 – 326

externalized configuration management

centralized configuration management with Spring Cloud Config Server 128 – 139

configuration server REST API 138 – 139

making configuration server resilient 137 – 138

setting up configuration server 133 – 137

using Git to store configuration data 131 – 133

configuration in Spring 115 – 124

profiles 121 – 124

properties 116 – 121

one build, multiple configurations 125 – 128

configuring application through command-line arguments 126

configuring application through environment variables 127 – 128

configuring application through JVM system properties 126 – 127

using configuration server with Spring Cloud Config Client 140 – 146

making configuration client resilient 142 – 143

refreshing configuration at runtime 143 – 146

setting up configuration client 140 – 142

ExternalSecret manifest 499

ExternalSecret object 499

External Secrets project 499

F

FaaS (Function as a Service) 28

fail-build property 109

fail-fast property 143

FAILURE_RATE_EXCEEDED event 310

fallback REST APIs 308

fallbacks 285 – 286

fallbackUri property 308

Farley, David 18 – 19, 95, 103

fat-JARs 76

fault tolerance 304 – 311

circuit breakers with Spring Cloud Circuit Breaker 306

combining circuit breakers, retries, and time limiters 309 – 311

configuring circuit breaker with Resilience4J 307

defining fallback REST APIs with Spring WebFlux 308

feature flags 121 – 124

Fifteen-Factor methodology. *See* 15-Factor methodology

filter operator 258

filters 303 – 304, 366

findAll() method 166

fixed connection pool 302

flatMap() operator 258, 270

Fluent Bit 452 – 455

Flux 257 – 258

Flux<Order> object 269, 271

Flux publisher 340

Flux<T> object 269

Flyway 173 – 177

evolving database with 176 – 177

initializing database schema with 175 – 176

managing database schemas with 267 – 269

migrations in Spring Boot 480 – 481

version control for database 173 – 175

writing Flyway migration to add new audit metadata to schema 435 – 436, 439

flyway Actuator endpoint 480 – 481

formLogin() method 368

Fowler, Martin 16

Fremantle, P. 4

FROM instruction 183

functional acceptance tests 523

Function as a Service (FaaS) 9, 28

FunctionCatalog object 342

Function function 335, 339, 567

functions 335 – 343

binding functions to message channels 346 – 351

configuring destination bindings 347 – 348

destination bindings 347

exploring exchanges and queues in RabbitMQ 349 – 351

composing and integrating functions 340 – 342

implementing business logic as 568 – 571

making messaging resilient to failures 354 – 355

using functional paradigm in 335 – 340

implementing business logic via functions 337 – 339

initializing Spring Cloud Function project 336 – 337

using imperative and reactive functions 339 – 340

writing integration tests with @FunctionalSpringBootTest 342 – 343

writing integration tests with test binder 352 – 354

FunctionsStreamIntegrationTests class 352

G

GDPR (General Data Protection Regulation) 7, 151

Genre enum 560

genreQuote function 571

GET /books/{bookIsbn} endpoint 277

getAllOrders() method 440 – 441

getBookByIsbn() method 281 – 282, 284 – 285

GET endpoint 271, 311

getUser() method 388

ghcr.io/thomasvitale/<image_name> personal image 187

ghcr.io/<your_github_username>/catalog-service:latest container image 201, 512

ghcr.io/<your_github_username>/<image_name> format 187

ghcr.io/<your_github_username>/my-java-image:1.0.0 image 188

ghcr.io hostname 187

Git, using to store configuration data 131 – 133

GitHub Actions

automating Kubernetes manifests validation with 247 – 248

implementing acceptance stage with 525 – 528

implementing commit stage with 105 – 110

implementing production stage with 541 – 547

publishing container images with 208 – 213

GitHub catalog-service repository 211, 527

GitHub Container Registry 185 – 188

GitHub quote-function repository 573

GitOps 547 – 554

GKE (Google Kubernetes Engine) 9 – 10

GraalVM

compiling native executables with 563 – 564

overview 556 – 557

support for Spring Boot with Spring Native 557 – 563

bootstrapping new project with Spring Native and Spring Reactive Web 558 – 559

implementing business logic 559 – 560

implementing web controller 561

running and testing on JVM 562 – 563

writing integration tests 561 – 562

graceful degradation 279

graceful shutdown 39

grace period 237

Gradle 73 – 75

gradle build command 74

gradlew wrapper script 74

Grafana

managing logs with 452 – 455

managing traces with 474 – 475

monitoring metrics with 467 – 470

GrantedAuthority objects 420 – 422

green production environment 540

Gregory, Janet 95

Group parameter 45

grype tool 105, 587

H

half-open state 304

HandlerMapping component 79

HATEOAS (Hypermedia as the Engine of Application State) 94

headers 378

health endpoint 458

health probes 456, 460 – 464

configuring in Spring Boot and Kubernetes 460 – 464

configuring liveness and readiness probes in Kubernetes 462 – 464

customizing liveness and readiness probes 460 – 462

defining health probes for Spring Boot applications using Actuator 457 – 459

heapdump endpoint 483 – 484

heap dumps 483 – 484

Heckler, Mark 43

hedging 230

Helm chart 597 – 599

Herbst, N. R. 7

hexagonal architecture 87

High-Performance Java Persistence and SQL (Mihalcea) 156

HIPAA (Health Insurance Portability and Accountability Act) 151

Hoffman, Kevin 36

HomeController class 49, 120

horizontal scalability 11

Hossain, Monsur 396

HTTP 429 - Too Many Requests status 312, 314

HTTP Basic authentication strategy 488

HTTP GET endpoint 49

HTTP GET request 87 – 89, 138, 395, 458

HTTPie 587

httpie utility 570

HttpMessageConverter beans 90

HTTP port 80

HTTP POST request 89

HttpStatusServerEntryPoint 396

https tool 587

http tool 587

Humble, Jez 18, 95, 103

Hybrid cloud 8

hypermedia 94

Hypermedia as the Engine of Application State (HATEOAS) 94

I

IaaS (Infrastructure as a Service) 26

idempotency 355 – 358

identification 365

identity provider 374

ID Token 374, 409, 416 – 417

IEEE Cloud Computing (Davis) 10

ImagePullBackOff error status 227

images

customizing name and version 512 – 513, 532 – 533

Docker 52 – 54

immutable servers 17

imperative application 259

imperative functions 339 – 340

implementation coupling 328

info Actuator endpoint 481 – 482

info endpoint 481

info.system property 482

Infrastructure as a Service (IaaS) 8, 26

infrastructure as code 16

Ingress, defined 321

ingress add-on 324

Ingress API 320 – 323

Ingress Controller 320 – 323

ingress-nginx namespace 322

Ingress objects 323 – 326

InMemoryBookRepository class 86, 166

inner development loop 241

InputDestination bean 352

inspecting authenticated user context 386 – 390

instant startup time 566

integration profile 170 – 172

integration tests 96

with @SpringBootTest 97 – 99, 172 – 173, 425 – 428

with Spring Security 425 – 428

with Testcontainers 172 – 173, 425 – 428

writing 561 – 562

with @FunctionalSpringBootTest 342 – 343

with test binder 352 – 354

interactions 33, 65

issuer-uri property 418

J

JacksonTester utility 101

JARs 76 – 78

Java 582 – 583

Java Function interface 337

Java Message Service (JMS) API 331

Java parameter 46

Java Persistence with Spring Data and Hibernate (Tudose) 156

Java Runtime Environment (JRE) 17, 25, 74, 556

Java Stream API 257

Java Stream object 270

JAVA_TOOL_OPTIONS environment variable 476, 538

java—version command 181 – 183

Java Virtual Machine. See JVM

javax.persistence package 160

JdbcAggregateTemplate lower-level object 171

JIT (just-in-time) compilation 556

JMS (Java Message Service) API 331

jobs, defined 106

JOSE Header 378

JRE (Java Runtime Environment) 17, 25, 74, 556

JSON serialization 101 – 103

JUnit 5 96 – 97

just-in-time (JIT) compilation 556

JVM heap memory 483

JVM (Java Virtual Machine) 528, 556

configuring application through 126 – 127

configuring resources for 537 – 538

running and testing on 562 – 563

JVM Runtime mode 557

JWA (JSON Web Algorithms) 379

JWE (JSON Web Encryption) 379

JWK (JSON Web Key) format 418

JWK Set URI 418

JWS (JSON Web Signature) 378 – 379

JWT authentication 419

JwtAuthenticationConverter bean 421

JwtDecoder bean 418, 424

JWT (JSON Web Token) 378

defining security policies for 430 – 431

defining security policies for JWT authentication 419 – 420

exchanging user information with 378 – 379

role-based access control with 420 – 423

defining authorization policies based on user roles 422 – 423

extracting user roles from Access Token 421 – 422

K

k3d option 585

key/value pairs 494

key/value pairs. *See* properties

Keycloak

configuring access to user roles in 413 – 415

configuring integration between Spring Security and 383 – 384, 418 – 419, 429

configuring user logout in 390 – 393

managing user accounts with 370 – 374

defining security realm 372

managing users and roles 373 – 374

registering application in 379 – 382

running using Helm chart 597 – 599

keycloak-issuer-resourceserver-secret keycloak 530

keycloak-system namespace 598

KeyResolver bean 315

kind option to install Kubernetes 585

kind section, of Kubernates manifest 223 – 224

Knative 574 – 580

deploying applications with Knative CLI 575 – 577

deploying applications with Knative manifests 577 – 580

setting up local Knative platform 575

Knative CLI 575 – 577, 588

Knative Eventing 574

Knative in Action (Chester) 580

Knative Serving 574

Kounev R. 7

Kratzke, N. 27

kubectl api-resources command 223

kubectl apply command 220

kubectl command 227

kubectl delete deployment <name> Kubernates CLI command 586

kubectl delete -f services command 221

kubectl delete pod <name> Kubernates CLI command 587

kubectl delete svc <service> Kubernates CLI command 587

kubectl describe pod <pod_name> command 228

kubectl explain <object_name> command 223, 225

kubectl get deployment Kubernates CLI command 586

kubectl get pod Kubernates CLI command 586

kubectl get pod -l app=catalog-service 515

kubectl get svc Kubernates CLI command 586

kubectl logs <pod_id> Kubernates CLI command 586

kubectl port-forward command 235, 324, 496, 539

kubectl port-forward svc <service> <host-port> Kubernates CLI command 587

kube-proxy proxy 232

Kubernetes

configuring applications on 487 – 491

disabling Spring Cloud Config 491

managing secrets with Spring Cloud Config 490 – 491

refreshing configuration at runtime with Spring Cloud Bus 488 – 490

securing configuration server with Spring Security 488

configuring health probes in 460 – 464

configuring liveness and readiness probes in Kubernetes 462 – 464

customizing liveness and readiness probes 460 – 462

configuring Prometheus metrics in 470 – 472

deleting all cloud resources 600

deployment pipeline 245 – 248

automating Kubernetes manifests validation with GitHub Actions 247 – 248

validating Kubernetes manifests in commit stage 246 – 247

Deployments for Spring Boot 221 – 228

controlling Pods with Deployments 222 – 223

creating Deployment for Spring Boot application 223 – 228

from containers to Pods 221

local Kubernetes development with Tilt 241 – 245

inner development loop with Tilt 241 – 243

visualizing Kubernetes workloads with Octant 244 – 245

managing configuration for multiple environments with Kustomize 506 – 509

managing containers with 57 – 62

deployments, pods, and services 58 – 59

running Spring application on Kubernetes 59 – 62

moving from Docker to 216 – 221

managing data services in local cluster 220 – 221

working with local Kubernetes cluster 218 – 219

running Keycloak using Helm chart 597 – 599

running Kubernetes cluster on DigitalOcean 590 – 593

running Polar UI 599

running PostgreSQL database on DigitalOcean 593 – 595

running RabbitMQ using Kubernetes Operator 596 – 597

running Redis on DigitalOcean 595 – 596

scalability and disposability 236 – 240

fast startup 236

graceful shutdown 236 – 239

scaling Spring Boot applications 239 – 240

service discovery and load balancing 228 – 236

client-side service discovery and load balancing 229 – 231

exposing Spring Boot applications with Kubernetes Services 233 – 236

overview 228 – 229

server-side service discovery and load balancing 231 – 233

setting up 585 – 587

using ConfigMaps and Secrets in 491 – 501

configuring Spring Boot with ConfigMaps 492 – 497

refreshing configuration at runtime with Spring Cloud Kubernetes 499 – 501

storing sensitive information with Secrets 497 – 499

Kubernetes Ingress 320 – 326

Ingress API and Ingress Controller 320 – 323

working with Ingress objects 323 – 326

kubernetes profile 122

Kubernetes Services 233 – 236

creating Service object from manifest 234 – 236

defining Service manifest with YAML 234

kubeval 247, 588

Kustomization base 511 – 512, 528, 530 – 531

Kustomization overlay 539

Kustomization resource 502, 505 – 506

Kustomize 501 – 516

customizing ConfigMaps 511 – 512

customizing environment variables 510 – 511

customizing image name and version 512 – 513

customizing number of replicas 514 – 516

defining configuration overlay for staging 509 – 510

managing Kubernetes configuration for multiple environments with 506 – 509

using to manage and configure Spring Boot applications 502 – 506

L

label() function 335, 339, 341, 352

label() method 340

{label} parameter 132, 138

label action 335

LABEL instruction 183

labels 133, 467

Language parameter 45

latest tag 187, 201, 512, 519, 521 – 522

layered-JAR mode 194

layers 181

Learn Docker in a Month of Lunches (Stoneman) 207

limits, assigning to container 535 – 536

Linkerd service mesh 221

list command 583

liveness probes

configuring in Kubernetes 462 – 464

customizing 460 – 462

liveness state 460

LoadBalancer Service approach 321

LoadBalancer type 321

load balancing 228 – 236

client-side service discovery and 229 – 231

exposing Spring Boot applications with Kubernetes Services 233 – 236

creating Service object from manifest 234 – 236

defining Service manifest with YAML 234

overview 228 – 229

server-side service discovery and 231 – 233

load-balancing strategy 229

local Kubernetes cluster

managing data services in 220 – 221

working with 218 – 219

local Kubernetes development 241 – 245

inner development loop with Tilt 241 – 243

visualizing Kubernetes workloads with Octant 244 – 245

log aggregation/analytics 15

log aggregator 39

log collector 452

logger class 449

logging 448 – 455

15-Factor methodology 38 – 39

managing logs with Loki, Fluent Bit, and Grafana 452 – 455

with Spring Boot 449 – 451

adding logs to Spring Boot applications 450 – 451

configuring logging in 449 – 450

logging.pattern configuration property group 450

logout() configuration 392

Loki 452 – 455

Long object 338 – 339

lookup 229

loose coupling 12 – 13

M

macroservices 31

main() method 48, 557

manageability 15

management.endpoints.web.exposure.include property 480

manifests 246

automating Kubernetes validation with GitHub Actions 247 – 248

creating Deployment object from 227 – 228

creating Service object from 234 – 236

defining with YAML 224 – 227

deploying applications with Knative 577 – 580

map() operator 258, 270, 278

mappings 414

MartinFowler.com blog 338

Maven 73 – 75

MDC (Mapped Diagnostic Context) tool 451

memory 534 – 538

assigning resource requests and limits to container 535 – 536

configuring resources for JVM 537 – 538

optimizing for Spring Boot applications 536 – 537

message brokers 331 – 335

AMQP for messaging systems 332

using RabbitMQ for publish/subscribe communications 333 – 335

Message objects 353

metadata

adding audit metadata for users who created or updated JDBC data entity 434 – 435

adding audit metadata for users who created or updated R2DBC data entity 438 – 439

writing Flyway migration to add new audit metadata to schema 435 – 436

metadata section, of Kubernates manifest 223

metrics 464 – 472

configuring Prometheus metrics in Kubernetes 470 – 472

configuring with Spring Boot Actuator and Micrometer 465 – 467

monitoring with Prometheus and Grafana 467 – 470

metrics endpoint 465 – 466

Micrometer 465 – 467

microservices architectures 30 – 32

Microservices Patterns (Richardson) 30, 355

Miell, Ian 207

migrations 173

Mihalcea, Vlad 156

minikube 216, 218

minikube delete—profile polar command 219

minikube ip—profile polar command 404

minikube option 585

minikube stop command 218

minikube tunnel—profile polar command 324, 404

MockMvc object 97

mockOidcLogin() configuration object 401

MOCK Web environment option 98

mock web server 287 – 289

modularization 12

modular monolith 12

monitoring 14, 464 – 472, 479 – 484

configuring metrics with Spring Boot Actuator and Micrometer 465 – 467

configuring Prometheus metrics in Kubernetes 470 – 472

exposing application information 481 – 482

Flyway migrations in Spring Boot 480 – 481

generating and analyzing heap dumps 483 – 484

metrics with Prometheus and Grafana 467 – 470

Mono 257 – 258

Mono API 339

Mono<Book> object 287

monolith strangler 295

Monolith to Microservices (Newman) 328

Mono<Mono<Order>> object 270

Mono<Order> object 269 – 270

Mono<T> object 269 – 270

More Agile Testing (Crispin and Gregory) 95

mTLS (mutual Transport Layer Security) 221

multi-factor authentication 366

multi-stage builds 194

multi-tiered architectures 30 – 32

MY_ENV_VAR variable 127

my-java-image Docker image 187

my-java-image entry 187

N

Name parameter 45

namespaces 24, 187, 322

native executable 557

native-image GraalVM component 563

Native Image mode 557

native images 563 – 565

compiling native executables with GraalVM 563 – 564

containerizing with Buildpacks 564 – 565

natural key 159

networks optional root-level section 203

Newman, Sam 30, 328, 355

nodes 58

non-blocking architectures 255 – 259

from thread-per-request to event loop 255 – 257

Project Reactor 257 – 258

Spring reactive stack 258 – 259

non-blocking backpressure 257

NONE option 98

non-functional acceptance tests 524

non-reactive application 259

NOT operator 301

NOT_PERMITTED event 310

Nygard, Michael T. 254

O

OAuth2 423 – 428

authorization with 407 – 417

customizing tokens and propagating user roles 412 – 417

token relay from Spring Cloud Gateway to other services 409 – 412

integration tests with @SpringBootTest, Spring Security, and Testcontainers 425 – 428

protecting APIs with OAuth2 (imperative) 417 – 428

role-based access control with Spring Security and JWT 420 – 423

securing Spring Boot as OAuth2 Resource Server 417 – 420

testing OAuth2 with Spring Security and Testcontainers 423 – 428

protecting APIs with OAuth2 (reactive) 429 – 433

securing Spring Boot as OAuth2 Resource Server 429 – 431

testing OAuth2 with Spring Security and Testcontainers 431 – 433

testing secured REST controllers with @WebMvcTest and Spring Security 423 – 425

OAuth2AuthorizedClient objects 411 – 412

OAuth 2 in Action (Richer and Sanso) 375

OAuth2 Resource Server 417 – 420, 429 – 431

configuring integration between Spring Security and Keycloak 418 – 419, 429

defining security policies for JWT authentication 419 – 420, 430 – 431

ObjectMapper utility 353

object-relational mapping (ORM) framework 156

observability

application management and monitoring with Spring Boot Actuator 479 – 484

exposing application information 481 – 482

generating and analyzing heap dumps 483 – 484

monitoring Flyway migrations in Spring Boot 480 – 481

distributed tracing with OpenTelemetry and Tempo 472 – 479

configuring tracing in Spring Boot with OpenTelemetry 475 – 479

managing traces with Tempo and Grafana 474 – 475

health probes with Spring Boot Actuator and Kubernetes 456 – 464

configuring 460 – 464

defining health probes for Spring Boot applications using Actuator 457 – 459

logging with 448 – 455

managing logs with Loki, Fluent Bit, and Grafana 452 – 455

Spring Boot 449 – 451

metrics and monitoring with 464 – 472

configuring metrics with Spring Boot Actuator and Micrometer 465 – 467

configuring Prometheus metrics in Kubernetes 470 – 472

monitoring metrics with Prometheus and Grafana 467 – 470

Polar Bookshop 66

properties of cloud native applications 14 – 15

OCI (Open Container Initiative) 26, 52, 583

Octant 244 – 245, 588

octant command 539

OidcClientInitiatedServerLogoutSuccessHandler class 390, 392

oidcLogoutSuccessHandler() method 392

OIDC (OpenID Connect) 374

OIDC Provider 376

OidcUser class 388, 401, 409, 416

OidcUser type 387, 441

onError() operator 273

onErrorResume() operator 285

OOMKilled (OutOfMemory killed) status 536

Open Container Initiative (OCI) 26, 52, 583

authenticating users with 374 – 378

authentication with 382 – 393

adding new dependencies 383

basic Spring Security configuration 384 – 385

configuring integration between Spring Security and Keycloak 383 – 384

configuring user logout in Spring Security and Keycloak 390 – 393

inspecting authenticated user context 386 – 390

testing 400 – 404

testing CSRF (Cross-Site Request Forgery) 402 – 404

testing OIDC authentication 400 – 402

testing authentication 400 – 402

OpenID Connect in Action (Siriwardena) 375

openid scope 413

open state 304 – 305

OpenTelemetry 472 – 479

configuring tracing in Spring Boot with 475 – 479

managing traces with Tempo and Grafana 474 – 475

OpenTelemetry (OTel) 473

OpenTelemetry protocol (OTLP) 477

Operator 596 – 597

optimistic locking 159

Optional<Customer> objects 258

orchestration 27 – 28

Order() constructor 439, 442

order-accepted.dispatcher-service queue 348 – 350

OrderAcceptedEvent message 362

order-accepted exchange 349 – 351

OrderAcceptedMessage object 351

orderAcceptedMessage object 359

OrderAcceptedMessage record 337, 358

Order all-args constructor 439

OrderController class 271

OrderControllerWebFluxTests class 291

OrderDispatchedEvent message 362

order-dispatched exchange 349, 351

OrderDispatchedMessage class 356

OrderDispatchedMessage objects 356

OrderDispatchedMessage record 339

order-dispatched.order-service queue 358

order-dispatched queue 351

Order domain entity 271

OrderFunctions class 356

Order Java record 265

Order object 286, 359

Order record 265 – 266, 438

OrderRepository behavior 289

OrderRepository interface 267

OrderRepositoryR2dbcTests class 289, 441

OrderRequest DTO class 271

OrderRequest object 271

OrderRequest record 271

Order Service 264

OrderServiceApplication class 274

OrderServiceApplicationTests class 362, 441

OrderService class 269 – 270, 278, 291, 356 – 357, 359 – 360, 439

order-service consumer group 358

order-service Git repository 261

OrderStatus enum 266

Order type 270

org.springframework.data.annotation package 158 – 159

org.springframework.transaction.annotation package 168

ORM (object-relational mapping) framework 156

OR operator 301

OS container 24, 26

OTEL_EXPORTER_OTLP_ENDPOINT environment variable 476

OTel (OpenTelemetry) 473

OTEL_SERVICE_NAME environment variable 476

OTLP (OpenTelemetry protocol) 477

OutOfMemoryError error 483

OutOfMemory killed (OOMKilled) status 536

OutputDestination bean 353

overlays

defining configuration overlay for production 528 – 534

customizing ConfigMaps 531

customizing environment variables 529

customizing image name and version 532 – 533

customizing number of replicas 533 – 534

customizing Secrets and volumes 530 – 531

defining configuration overlay for staging 509 – 510

P

pack() + label() composed function 342

pack() function 339, 341, 352

pack() method 338

pack| function 351

pack action 335

Package name parameter 45

packages 188, 597

Packaging parameter 45

packaging Spring Boot applications 188 – 201

building container images for production 193 – 197

Dockerfiles or Buildpacks 197

performance 193 – 195

security 195 – 196

containerizing Spring Boot with Cloud Native Buildpacks 197 – 201

containerizing Spring Boot with Dockerfiles 191 – 193

preparing Spring Boot for containerization 189 – 191

exposing application services through port forwarding 189

using Docker's built-in DNS server for service discovery 189 – 191

pack function 335

packlabel-in-0 binding 349, 352

packlabel-out-0 binding 349 – 350, 353

Parnas, D. L. 12

Password Grant 426

patches 506

Path predicate 300 – 301

PAT (personal access token) 186, 543

payload 378

Peinl, R. 27

people gap 40

performance 193 – 195

performance property 151

permissions 106

persistent data. *See* data persistence

persistent entities

Spring Data R2DBC 265 – 267

with Spring Data 158 – 163

creating database schema 162 – 163

making domain class persistent 158 – 161

personal access token (PAT) 186, 543

phoenix servers 17

PID (process identifier) 450

pipe (|) operator 341

Platform as a Service (PaaS) 9

platforms 5

platform team 216

Pods

controlling with Deployments 222 – 223

Kubernetes 58 – 59

overview 221

Polar Bookshop 62 – 67

exploring patterns and technologies used in project 63 – 67

build and deployment 67

configuration 65 – 66

data 65

observability 66

resilience 66

routing 66

security 66 – 67

testing 67

UI 67

web and interactions 65

requirements of system 62 – 63

PolarBookshop realm 372, 381, 413

polar.catalog-service-uri property 273

polar cluster 219, 322

polardb_catalog database 264

polar-deployment repository 203, 220, 240 – 241, 245, 264, 313, 322 – 323, 333 – 334, 349, 371, 394, 404, 507, 509, 528, 542 – 546, 549, 552, 593

polar-deployment source code 544

polar.greeting bean 119

POLAR_GREETING environment variable 127

polar.greeting property 120 – 121, 124, 126 – 128, 131 – 132, 141, 145, 504, 511, 515, 551

polar-keycloak:8080 container name 442

polar-keycloak-client-credentials 598

polar-keycloak container 371, 414

polar-keycloak hostname 404

polar-keycloak Keycloak Service name 442

polar-keycloak-secret shared secret 384

polar-keycloak service 381

polar-postgres-catalog-credentials database 530

polar-postgres container name 190, 193

polar-postgres server 593

PolarProperties bean 118, 145 – 146

polar-rabbitmq-credentials 597

polar-redis server 595

polar.testdata.enabled property 124

Polar UI 599

port binding, 15-Factor methodology 40

port forwarding 53, 189

port mapping 53, 189

port publishing 189

post() method 91

POSTGRES_DB environment variable 264

PostgreSQL 157, 262

PostgreSQL database

running as container 153 – **154**

running for Order Service 264

running on DigitalOcean 593 – **595**

postgresql image 187

predicates 299 – **303**

preStop hook 238

primary key 155

principal 387, **389**

privacy 440 – **441**

private cloud 7

process identifier (PID) 450

processors 335

prod profile 121 – **122**, **132**, **138**, **141**, **529**, **539**

producers 332, **335**

product flexibility 12

production stage 103, **540** – **547**

implementing with GitHub Actions 541 – **547**

overview 541

{profile} parameter 132, **138**

profiles 115, **121** – **124**, **218**

using as configuration groups 124

using as feature flags 121 – 124

Project parameter 45

Project Reactor 257 – 258

Prometheus

configuring metrics in Kubernetes 470 – 472

monitoring metrics with 467 – 470

prometheus Actuator endpoint 465 – 466

properties 5, 115 – 121

defining custom properties 118 – 120

loose coupling 12 – 13

manageability 15

observability 14 – 15

resilience 13

scalability 11 – 12

using application properties 117 – 118

using custom properties 120 – 121

Properties format 116 – 117

protocol network 233

public Clients 379 – 380

public cloud 8

public static void main(String[] args) method 48

publish/subscribe communications

model overview 329 – 331

using RabbitMQ for 333 – 335

Publisher/subscriber (pub/sub) model 329

publishers 258, 335

Publisher<T> objects 269

publishing images

on GitHub Container Registry 185 – 188

with GitHub Actions 208 – 213

publishOrderAcceptedEvent method 359 – 360

Pulled automatically principle 548

put() method 91

Q

QA (quality assurance) environment 24

quality attributes 524

queues 332, 349 – 351

QuoteController class 561

QuoteFunctions class 568

quote-function service 576

Quote record 559

QuoteServiceApplicationTests class 561

QuoteService class 560

quote-service Git repository 558

R

R2DBC (Reactive Relational Database Connectivity) 263

RabbitMQ 331 – 335

AMQP for messaging systems 332

configuring Spring integration with 344 – 346

exploring exchanges and queues in 349 – 351

running using Kubernetes Operator 596 – 597

using for publish/subscribe communications 333 – 335

rabbitmq-system namespace 597

RANDOM_PORT Web environment 98

RateLimiterConfig class 315, 389

rate limiting 311 – 316

configuring request rate limiter 314 – 316

integrating Spring with Redis 313 – 314

running Redis as container 312 – 313

RBAC (role-based access control) 373, 407 – 408, 421

RDS (Relational Database Service) 151

reactive applications 286 – 292

testing data persistence with @DataR2dbcTest and Testcontainers 289 – 290

testing REST clients with mock web server 287 – 289

testing REST controllers with @WebFluxTest 290 – 292

ReactiveAuditorAware bean 438

ReactiveClientRegistrationRepository bean 393

reactive clients 273 – 279

how to exchange data 275 – 276

implementing REST clients with WebClient 276 – 279

service-to-service communication in Spring 273 – 274

reactive functions 339 – 340

ReactiveJwtDecoder bean 432

Reactive Manifesto 255

Reactive Relational Database Connectivity (R2DBC) 263

reactive repositories 267

reactive servers 259 – 272

bootstrapping reactive application with Spring Boot 261 – 263

exposing REST API with Spring WebFlux 271 – 272

implementing business logic with reactive streams 269 – 270

persisting data reactively with Spring Data R2DBC 263 – 269

connecting to database with R2DBC 265

defining persistent entities 265 – 267

managing database schemas with Flyway 267 – 269

running PostgreSQL database for Order Service 264

using reactive repositories 267

asynchronous and non-blocking architectures with 255 – 259

from thread-per-request to event loop 255 – 257

Project Reactor 257 – 258

Spring reactive stack 258 – 259

reactive clients with Spring WebClient 273 – 279

how to exchange data 275 – 276

implementing REST clients with 276 – 279

service-to-service communication in Spring 273 – 274

reactive servers with Spring WebFlux and Spring Data R2DBC 259 – 272

bootstrapping reactive application with Spring Boot 261 – 263

exposing REST API with Spring WebFlux 271 – 272

implementing business logic with reactive streams 269 – 270

persisting data reactively with Spring Data R2DBC 263 – 269

resilient applications with 279 – 286

fallbacks and error handling 285 – 286

retries 283 – 285

timeouts 280 – 282

testing reactive applications with Spring, Reactor, and Testcontainers 286 – 292

testing data persistence with @DataR2dbcTest and Testcontainers 289 – 290

testing REST clients with mock web server 287 – 289

testing REST controllers with @WebFluxTest 290 – 292

reactive streams

implementing business logic with 269 – 270

with Mono and Flux 257 – 258

Reactor Test 262

readiness probes

configuring in Kubernetes 462 – 464

customizing 460 – 462

readiness state 460

read timeout 280

realm roles mapper 414

realms 372

Real-World Cryptography (Wong) 418

Redis

distributed session management with 316 – 320

request rate limiting with 311 – 316

configuring request rate limiter 314 – 316

integrating Spring with Redis 313 – 314

running Redis as container 312 – 313

running on DigitalOcean 595 – 596

storing Access Tokens in 411 – 412

reduced memory consumption 566

refresh endpoint 458

refreshing configuration 143 – 146

changing configuration at runtime 145 – 146

enabling configuration refresh 145

RefreshScopeRefreshedEvent event 145 – 146, 488

Refresh Token 410

registrationId identifier 384

Relational Database Service (Amazon RDS) 151

relaxed binding 127

releasable state 524

release candidates 103, 207, 246, 519

building in commit stage 207 – 208

versioning for continuous delivery 519 – 522

Release It! Design and Deploy Production-Ready Software (Nygard) 254

release stage, 15-Factor methodology 38

Relying Party (RP) 376

repeatable migrations 173

replenish rate 314

replicas 514 – 516, 533 – 534

ReplicaSet object 222

repositories 84, 155, 187

repository abstraction 85 – 86, 267

repository-dispatch action 543

Repository interfaces 166

repository pattern 166

RequestRateLimiter filter 314 – 315, 317

requests

assigning resource requests to container 535 – 536

processing through filters 303 – 304

rate limiting 311 – 316

configuring request rate limiter 314 – 316

integrating Spring with Redis 313 – 314

running Redis as container 312 – 313

Resilience4J 304 – 311

circuit breakers with Spring Cloud Circuit Breaker 306

combining circuit breakers, retries, and time limiters 309 – 311

configuring circuit breaker with Resilience4J 307

defining fallback REST APIs with Spring WebFlux 308

resilience property 151

resiliency

configuration server 137 – 138

making configuration client resilient 142 – 143

making messaging resilient to failures 354 – 355

Polar Bookshop 66

properties of cloud native applications 13

reasons for choosing cloud native 22 – 23

with Reactive Spring 279 – 286

fallbacks and error handling 285 – 286

retries 283 – 285

timeouts 280 – 282

resource limits 534

resource manifests 59, 246

Resource Owner 375, 408

Resource Server 407 – 408

resources.requests.cpu request 536

resources.requests.memory request 537

responses 303 – 304

REST APIs

configuration server 138 – 139

fallback, defining with Spring WebFlux 308

REST clients

implementing with Spring WebClient 276 – 279

testing with mock web server 287 – 289

REST controllers

testing secured 423 – 425, 432 – 433

testing with @WebFluxTest 290 – 292

REST endpoints 340 – 342

RESTful applications

building with Spring MVC 82 – 94

data validation and error handling 90 – 93

evolving APIs for future requirements 94

implementing REST API with Spring MVC 87 – 90

REST API first, business logic later 82 – 87

testing with Spring 94 – 103

integration tests with @SpringBootTest 97 – 99

testing JSON serialization with @JsonTest 101 – 103

testing REST controllers with @WebMvcTest 100 – 101

unit tests with JUnit 5 96 – 97

RestTemplate HTTP request 273

retries 283 – 285

combining circuit breakers and time limiters 309 – 311

for WebClient 284

how to use effectively 284 – 285

Retry filter 303 – 304, 308, 311

retry pattern 142

retryWhen() operator 258, 273, 284, 354

Richardson, Chris 30, 355

Richer, Justin 375

role-based access control 420 – 423

defining authorization policies based on user roles 422 – 423

extracting user roles from Access Token 421 – 422

role-based access control (RBAC) 373, 407 – 408, 421

ROLE_ prefix 421

roles

defining authorization policies based on 422 – 423

managing with Keycloak 373 – 374

with Spring Cloud Gateway and OAuth2 407 – 417

customizing tokens and propagating user roles 412 – 417

token relay from Spring Cloud Gateway to other services 409 – 412

roles claim 388, 413 – 417, 421

roles list 389

roles scope 413 – 415

rolling restart 499, 504, 515

rolling update 540

rolling upgrade 177

RouterFunction<ServerResponse> bean 308

routes 66, 299 – 303

RP (Relying Party) 376

RUN instruction 183

running application

15-Factor methodology stage 38

building Spring Boot application 50 – 51

S

SaaS (Software as a Service) 9

saga pattern 360

Salatino, Mauricio 580, 590

Sanchez, Illary Huaylupo 30

Sanso, Antonio 375

save() method 166, 270

saveAll() method 168

Sayers, Aidan Hobson 207

scalability

Kubernetes 236 – 240

fast startup 236

graceful shutdown 236 – 239

scaling Spring Boot applications 239 – 240

properties of cloud native applications 11 – 12

scalability property 150

scale 23

scaling to zero 29, 566

scheduling, in container orchestration 58

scopes 413

SealedSecret object 498

Sealed Secrets project 498

Secrets 491 – 501

configuring Spring Boot with ConfigMaps 492 – 497

customizing 530 – 531

defined 497

refreshing configuration at runtime with Spring Cloud Kubernetes 499 – 501

storing sensitive information with 497 – 499

with Spring Cloud Config 490 – 491

security

authenticating users with Spring Security and OpenID Connect 382 – 393

adding new dependencies 383

basic Spring Security configuration 384 – 385

configuring integration between Spring Security and Keycloak 383 – 384

configuring user logout in Spring Security and Keycloak 390 – 393

inspecting authenticated user context 386 – 390

authentication with 374 – 382

authenticating users with OpenID Connect 374 – 378

exchanging user information with JWT (JSON Web Token) 378 – 379

registering application in Keycloak 379 – 382

authorization and roles with Spring Cloud Gateway and OAuth2 407 – 417

customizing tokens and propagating user roles 412 – 417

token relay from Spring Cloud Gateway to other services 409 – 412

building container images 195 – 196

data 433 – 443

auditing with Spring Security and Spring Data JDBC 433 – 436

protecting user data with Spring Security and Spring Data R2DBC 437 – 441

testing data auditing and protection with @WithMockUser and Spring Data R2DBC 441 – 443

testing data auditing with Spring Data and @WithMockUser 436 – 437

integrating Spring Security with SPAs (single-page applications) 393 – 400

controlling authentication flow 395 – 398

protecting against CSRF (Cross-Site Request Forgery) 398 – 400

running Angular application 393 – 395

managing user accounts with Keycloak 370 – 374

defining security realm 372

managing users and roles 373 – 374

Polar Bookshop 66 – 67

protecting APIs with Spring Security and OAuth2 (imperative) 417 – 428

role-based access control with Spring Security and JWT 420 – 423

securing Spring Boot as OAuth2 Resource Server 417 – 420

testing OAuth2 with Spring Security and Testcontainers 423 – 428

protecting APIs with Spring Security and OAuth2 (reactive) 429 – 433

securing Spring Boot as OAuth2 Resource Server 429 – 431

testing OAuth2 with Spring Security and Testcontainers 431 – 433

Spring Security fundamentals 366 – 369

testing Spring Security and OpenID Connect 400 – 404

testing CSRF (Cross-Site Request Forgery) 402 – 404

testing OIDC authentication 400 – 402

SecurityConfig class 367 – 368, 385, 392, 397, 399, 412, 419, 421, 430, 457, 461

SecurityConfigTests class 402

SecurityContext object 387, 389, 433, 437

SecurityWebFilterChain bean 367, 384, 392, 397

selector label 233

self-contained JARs 77

semantic versioning 520

ServerHttpSecurity object 367 – 368, 384

serverless applications 566 – 574

build and publish 571 – 573

building with Spring Cloud Function 567 – 571

bootstrapping new project with Spring Native and Spring Cloud Function 567 – 568

implementing business logic as functions 568 – 571

composing and integrating functions 340 – 342

deploying on cloud 573 – 574

deploying with Knative 574 – 580

deploying applications with Knative CLI 575 – 577

deploying applications with Knative manifests 577 – 580

setting up local Knative platform 575

overview 28 – 29

server.netty.connection-timeout property 263, 280

server.netty.idle-timeout property 263

ServerOAuth2AuthorizedClientRepository 411 – 412

server.port property 80, 116 – 117, 263

server.shutdown property 237

server-side rate limiters 312

server-side service discovery 231 – 233

server.tomcat.connection-timeout property 80

server.tomcat.keep-alive-timeout property 81

server.tomcat.threads.max property 81

service-based architecture 32 – 33

application services 32

data services 33

interactions 33

service discovery 228 – 236

client-side service discovery and load balancing 229 – 231

exposing Spring Boot applications with Kubernetes Services 233 – 236

creating Service object from manifest 234 – 236

defining Service manifest with YAML 234

overview 228 – 229

server-side service discovery and load balancing 231 – 233

Service object 496

service registry 229

services 58 – **59**, **83**, **577**

services section 226

service-to-service communication 273 – **274**

Servlet container 26, **76** – **77**

Servlet Filter class 366

setPostLogoutRedirectUri() method 390, **392**

setter-based dependency injection 85

share-nothing architecture 41

shutdown, Kubernetes 236 – **239**

SIGKILL signal 237

signature, JWT 379

SIGTERM signal 237 – **238**

silos 19

single-page applications. *See* SPAs

sinks 335

Siriwardena, Prabath 375

SLF4J (Simple Logging Facade for Java) 449

slice tests 100, **171**

snapshot-dependencies layer 194

snowflake servers 17

Software as a Service (SaaS) 9 – 10

sources 335

spans 472

SPAs (single-page applications) 393 – 400

controlling authentication flow 395 – 398

protecting against CSRF 398 – 400

running Angular application 393 – 395

SPA_URL environment variable 394 – 395

spec section, of deployment manifest 225

speed 21 – 22

Spilcă, Laurențiu 43

Spring 42 – 51

building Spring Boot application 43 – 51

bootstrapping application 48 – 49

exploring build configuration 47 – 48

implementing controller 49 – 50

initializing project 44 – 47

running application 50 – 51

testing application 50

configuration in 115 – 124

profiles 121 – 124

properties 116 – 121

integrating with Redis 313 – 314

landscape 42 – 43

running as container 55 – 56

running on Kubernetes 59 – 62

testing RESTful application with 94 – 103

integration tests with @SpringBootTest 97 – 99

testing JSON serialization with @JsonTest 101 – 103

testing REST controllers with @WebMvcTest 100 – 101

unit tests with JUnit 5 96 – 97

spring.application.name property 132, 141, 263, 467, 476

Spring Boot 42, 556 – 565

bootstrapping reactive application with 261 – 263

building application 43 – 51

bootstrapping application 48 – 49

exploring build configuration 47 – 48

implementing controller 49 – 50

initializing project 44 – 47

running application 50 – 51

testing application 50

compiling as native images 563 – 565

compiling native executables with GraalVM 563 – 564

containerizing native images with Buildpacks 564 – 565

configuring for production 528 – 540

configuring CPU and memory for Spring Boot containers 534 – 538

defining configuration overlay for production 528 – 534

deploying Spring Boot in production 539 – 540

configuring tracing in 475 – 479

configuring with ConfigMaps 492 – 497

deployment pipeline 207 – 213

building release candidates in commit stage 207 – 208

publishing container images with GitHub Actions 208 – 213

Deployments for 221 – 228

controlling Pods with Deployments 222 – 223

creating Deployment for Spring Boot application 223 – 228

from containers to Pods 221

Flyway migrations in 480 – 481

GraalVM and native images 556 – 557

GraalVM support with Spring Native 557 – 563

bootstrapping new project with Spring Native and Spring Reactive Web 558 – 559

implementing business logic 559 – 560

implementing web controller 561

running and testing on JVM 562 – 563

writing integration tests 561 – 562

logging with 449 – 451

adding logs to Spring Boot applications 450 – 451

configuring 449 – 450

managing Spring Boot containers with Docker Compose 202 – 207

debugging Spring Boot containers 204 – 207

using Docker Compose to manage container life cycle 203 – 204

packaging applications as container images 188 – 201

building container images for production 193 – 197

containerizing Spring Boot with Cloud Native Buildpacks 197 – 201

containerizing Spring Boot with Dockerfiles 191 – 193

preparing Spring Boot for containerization 189 – 191

securing as OAuth2 Resource Server 417 – 420, 429 – 431

configuring integration between Spring Security and Keycloak 418 – 419, 429

defining security policies for JWT authentication 419 – 420, 430 – 431

working with container images on Docker 180 – 188

container images, overview of 181 – 182

creating images with Dockerfiles 182 – 185

publishing images on GitHub Container Registry 185 – 188

Spring Boot: Up & Running (Heckler) 43

Spring Boot Actuator

application management and monitoring with 479 – 484

exposing application information 481 – 482

generating and analyzing heap dumps 483 – 484

monitoring Flyway migrations in Spring Boot 480 – 481

configuring health probes in 460 – 464

configuring liveness and readiness probes in Kubernetes 462 – 464

customizing liveness and readiness probes 460 – 462

configuring metrics with 465 – 467

defining health probes for 457 – 459

Spring Boot applications

exposing with Kubernetes Services 233 – 236

creating Service object from manifest 234 – 236

defining Service manifest with YAML 234

scaling 239 – 240

using Kustomize to manage and configure 502 – 506

spring-boot-loader layer 194

Spring Boot parameter 45

spring-boot-starter-test dependency 96 – 97

spring-boot-starter-web dependency 78

springBoot task 481

Spring Boot Test 48, 135, 262, 298, 337

Spring Cloud Bus 488 – 490

Spring Cloud Circuit Breaker 304 – 311

circuit breakers with 306

combining circuit breakers, retries, and time limiters 309 – 311

configuring circuit breaker with Resilience4J 307

defining fallback REST APIs with Spring WebFlux 308

Spring Cloud Config

disabling 491

managing secrets with 490 – 491

Spring Cloud Config Client

using configuration server with 140 – 146

making configuration client resilient 142 – 143

refreshing configuration at runtime 143 – 146

setting up configuration client 140 – 142

spring.cloud.config.fail-fast property 143

spring.cloud.config.request-connect-timeout property 142

spring.cloud.config.request-read-timeout property 142

spring.cloud.config.retry.multiplier property 143

Spring Cloud Config Server

centralized configuration management with 128 – 139

configuration server REST API 138 – 139

making configuration server resilient 137 – 138

setting up configuration server 133 – 137

using Git to store configuration data 131 – 133

spring.cloud.config.server.git.basedir property 137

spring.cloud.config.server.git.clone-on-start property 137

spring.cloud.config.server.git.default-label property 136

spring.cloud.config.server.git.force-pull property 137

spring.cloud.config.server.git.timeout property 137

spring.cloud.config.server.git.uri property 136

SPRING_CLOUD_CONFIG_URI environment variable 213, 248

spring.cloud.config.uri property 140

Spring Cloud Function 335 – 343

building serverless applications 567 – 571

bootstrapping new project with Spring Native and Spring Cloud Function 567 – 568

implementing business logic as functions 568 – 571

composing and integrating functions 340 – 342

using functional paradigm in 335 – 340

implementing business logic via functions 337 – 339

initializing Spring Cloud Function project 336 – 337

using imperative and reactive functions 339 – 340

writing integration tests with @FunctionalSpringBootTest 342 – 343

spring.cloud.function.definition configuration property 570

spring.cloud.function.definition property 341, 347, 573

Spring Cloud Gateway 295, 298 – 304

adopting token relay pattern in 411

authorization with 407 – 417

customizing tokens and propagating user roles 412 – 417

token relay from Spring Cloud Gateway to other services 409 – 412

bootstrapping edge server with 297 – 299

defining routes and predicates 299 – 303

processing requests and responses through filters 303 – 304

request rate limiting with 311 – 316

configuring request rate limiter 314 – 316

integrating Spring with Redis 313 – 314

running Redis as container 312 – 313

spring.cloud.gateway.httpclient.pool property group 302

spring.cloud.gateway.redis-rate-limiter property group 316

Spring Cloud GCP (Google Cloud Secret Manager) 490

Spring Cloud Kubernetes 499 – 501

Spring Cloud Kubernetes Config Server 500

Spring Cloud Kubernetes Configuration Watcher 499

Spring Cloud Kubernetes Discovery Server 233

Spring Cloud Stream

processing messages with 344 – 355

binding functions to message channels 346 – 351

configuring integration with RabbitMQ 344 – 346

making messaging resilient to failures 354 – 355

writing integration tests with test binder 352 – 354

producing and consuming messages with 355 – 363

implementing event consumers, and problem of idempotency 355 – 358

implementing event producers, and problem of atomicity 358 – 363

spring.config.additional-location 494

spring.config.import 495

spring.config.import property 140, 531

Spring container 26, 42

Spring Data

data repositories with 166 – 169

defining transactional contexts 168 – 169

using data repository 166 – 168

testing data auditing with 436 – 437

Spring Data JDBC 157, 433 – 436

adding audit metadata for users who created or updated JDBC data entity 434 – 435

data persistence with 154 – 169

connecting to database with JDBC 156 – 158

data repositories with Spring Data 166 – 169

defining persistent entities with Spring Data 158 – 163

enabling and configuring JDBC auditing 163 – 165

defining auditor to capture who created or updated JDBC data entity 433 – 434

writing Flyway migration to add new audit metadata to schema 435 – 436

Spring Data R2DBC 262 – 269, 437 – 441

adding audit metadata for users who created or updated R2DBC data entity 438 – 439

connecting to database with R2DBC 265

defining auditor to capture who created or updated R2DBC data entity 437 – 438

defining persistent entities 265 – 267

ensuring user data privacy 440 – 441

managing database schemas with Flyway 267 – 269

running PostgreSQL database for Order Service 264

testing data auditing and protection with 441 – 443

using reactive repositories 267

writing Flyway migration to add new audit metadata to schema 439

spring.datasource.hikari.connection-timeout connection timeout 158

spring.datasource.hikari.connection-timeout property 280

spring.datasource.hikari.maximum-pool-size pool 158

spring.datasource.url property 170, 192 – 193

Spring Environment object 127 – 128

Spring Framework 42

Spring in Action, sixth edition (Walls) 43, 286

spring.lifecycle.timeout-per-shutdown-phase property 237

Spring Microservices in Action (Carnell and Sanchez) 30

Spring MVC 82 – 94

data validation and error handling 90 – 93

evolving APIs for future requirements 94

implementing REST API with 87 – 90

REST API first, business logic later 82 – 87

Spring Native 557 – 563

bootstrapping project with 558 – 559, 567 – 568

implementing business logic 559 – 560

implementing web controller 561

running and testing on JVM 562 – 563

writing integration tests 561 – 562

spring.profiles.active property 123, 132

spring.r2dbc properties 265

Spring reactive stack 258 – 259

Spring Reactive Web 262, 558 – 559, 568

Spring Security 423 – 428

auditing data with 433 – 436

adding audit metadata for users who created or updated JDBC data entity 434 – 435

defining auditor to capture who created or updated JDBC data entity 433 – 434

writing Flyway migration to add new audit metadata to schema 435 – 436

authentication with 382 – 393

adding new dependencies 383

basic Spring Security configuration 384 – 385

configuring integration between Spring Security and Keycloak 383 – 384

configuring user logout in Spring Security and Keycloak 390 – 393

inspecting authenticated user context 386 – 390

configuring access to user roles in 415 – 416

fundamentals 366 – 369

integrating with SPAs 393 – 400

controlling authentication flow 395 – 398

protecting against CSRF 398 – 400

running Angular application 393 – 395

integration tests with 425 – 428

protecting APIs with 417 – 433

role-based access control with Spring Security and JWT 420 – 423

securing Spring Boot as OAuth2 Resource Server 417 – 420, 429 – 431

testing OAuth2 with Spring Security and Testcontainers 423 – 428, 431 – 433

protecting user data with 437 – 441

adding audit metadata for users who created or updated R2DBC data entity 438 – 439

defining auditor to capture who created or updated R2DBC data entity 437 – 438

ensuring user data privacy 440 – 441

writing Flyway migration to add new audit metadata to schema 439

role-based access control with 420 – 423

defining authorization policies based on user roles 422 – 423

extracting user roles from Access Token 421 – 422

securing configuration server with 488

testing 400 – 404

CSRF (Cross-Site Request Forgery) 402 – 404

OIDC authentication 400 – 402

testing secured REST controllers with 423 – 425

Spring Security Customizer interface 368

Spring Session Data Redis 317 – 320

spring.session.store-type property 319

spring.sql.init.mode property 175

Spring Start Here (Spilcaˇ) 43

Spring Web 48

Spring WebClient 273 – 279

how to exchange data 275 – 276

implementing REST clients with 276 – 279

retries for 284

service-to-service communication in Spring 273 – 274

timeouts for 281 – 282

Spring WebFlux

defining fallback REST APIs with 308

exposing REST API with 271 – 272

staging 509 – 510

staging profile 122, 510 – 511, 515

state 33, 148

stateless authentication 419

stateless processes, 15-Factor methodology 41

STATE_TRANSITION event 310

StepVerifier object 287

StepVerifier utility 289

Stoneman, Elton 207

strategic merge patch 510

StreamBridge object 359

Stream object 258

String type 472

submitOrder() method 360

subscribe() clause 357

subscribers 258

subscription 258

Supplier bean 360

Supplier function 335, 339, 567

surrogate key 159

System.getenv() method 127

T

tags 187, 467, 472

targetPort port 233

TDD (test-driven development) 19, 83, 95

technical key 159

telemetry, 15-Factor methodology 41

Tempo 472 – 479

configuring tracing in Spring Boot with OpenTelemetry 475 – 479

managing traces with 474 – 475

temporal coupling 328

Terraform in Action (Winkler) 590

test binder 352 – 354

TestChannelBinderConfiguration class 352

Testcontainers 262, 423 – 428

configuring for PostgreSQL 170 – 171

integration tests with 172 – 173, 425 – 428

testing data persistence with 171 – 172, 289 – 290

testing OAuth2 with Spring Security and 431 – 433

testing secured REST controllers with @WebMvcTest and Spring Security 423 – 425

test-credentials Secret object 497

testdata profile 122 – 123, 193, 515

test Gradle task 50

testing 400 – 404

CSRF (Cross-Site Request Forgery) 402 – 404

data auditing and protection with @WithMockUser and Spring Data R2DBC 441 – 443

data auditing with Spring Data and @WithMockUser 436 – 437

OAuth2 with Spring Security and Testcontainers 423 – 428, 431 – 433

integration tests with @SpringBootTest, Spring Security, and Testcontainers 425 – 428

testing secured REST controllers with @WebMvcTest and Spring Security 423 – 425

OIDC authentication 400 – 402

on JVM 562 – 563

Polar Bookshop 67

reactive applications 286 – 292

testing data persistence with @DataR2dbcTest and Testcontainers 289 – 290

testing REST clients with mock web server 287 – 289

testing REST controllers with @WebFluxTest 290 – 292

RESTful application with Spring 94 – 103

integration tests with @SpringBootTest 97 – 99

testing JSON serialization with @JsonTest 101 – 103

testing REST controllers with @WebMvcTest 100 – 101

unit tests with JUnit 5 96 – 97

Spring Boot application 50

writing integration tests 561 – 562

testing data persistence 169 – 173

configuring Testcontainers for PostgreSQL 170 – 171

integration tests with @SpringBootTest and Testcontainers 172 – 173

testing data persistence with @DataJdbcTest and Testcontainers 171 – 172

test profile 121

TestRestTemplate utility 97

test task 78

thread-per-request 78 – 80, 255 – 257

thread pool 81

Tilt 241 – 245, 588

inner development loop with 241 – 243

visualizing Kubernetes workloads with Octant 244 – 245

tilt up command 242

time gap 39

time limiters 309 – 311

timeout() operator 258, 273, 285

timeouts 280 – 282

for WebClient 281 – 282

how to use effectively 282

token-based authentication 419

token bucket algorithm 314

token relay pattern 410

tokens 412 – 417

configuring access to user roles in Keycloak 413 – 415

configuring access to user roles in Spring Security 415 – 416

extracting user roles from ID Token 416 – 417

token relay 409 – 412

adopting token relay pattern in Spring Cloud Gateway 411

storing Access Tokens in Redis 411 – 412

Tomcat 80 – 81

connection timeout 80 – 81

HTTP port 80

thread pool 81

tools 587 – 588

grype 587

HTTPie 587

Knative CLI 588

Kubeval 588

Octant 588

Tilt 588

tools gap 40

topic exchange 347

topologies 24 – **29**

containers 24 – **26**

orchestration 27 – **28**

serverless technologies 28 – **29**

trace 472

transactional contexts 168 – **169**

trust boundaries 412

Twelve-Factor methodology. *See* 15-Factor methodology

U

uber-JARs 77

ubuntu image 187

UI, Polar Bookshop 67

unit tests 96 – **97**

update operation 243

usability testing 524

UserController class 387 – **388**, **416**

UserControllerTests class 400

USER instruction 183

User model 386

User object 387 – **388**

users 375

authenticating with OpenID Connect 374 – **378**

authenticating with Spring Security and OpenID Connect 382 – 393

adding new dependencies 383

basic Spring Security configuration 384 – 385

configuring integration between Spring Security and Keycloak 383 – 384

configuring user logout in Spring Security and Keycloak 390 – 393

inspecting authenticated user context 386 – 390

exchanging user information with JWT (JSON Web Token) 378 – 379

managing with Keycloak 370 – 374

defining security realm 372

users and roles 373 – 374

propagating user roles 412 – 417

configuring access to user roles in Keycloak 413 – 415

configuring access to user roles in Spring Security 415 – 416

extracting user roles from ID Token 416 – 417

V

V3__Add_user_audit.sql migration script 435

validation 262

automating Kubernetes manifests 247 – 248

building RESTful application with Spring MVC 90 – 93

version control, Flyway 173 – 175

versioned migrations 173

VERSION environment variable 521

versioning 519 – 522

vertical scalability 11

visualizing 244 – 245

volumes 530 – 531

vulnerability database 105

W

Walls, Craig 43, 286

web

implementing web controller 561

Polar Bookshop 65

WebClient bean 276 – 277

WebClient object 276, 291

WebClientResponseException.NotFound exception 285

WebClient. RestTemplate HTTP request 273

WebEndpoints class 308

webEnvironment attribute 97

WebFilter bean 399

WebFilter class 366

web server 76 – 77

WebServerFactoryCustomizer bean 80

WebServerFactoryCustomizer<NettyReactiveWebServerFactory> component 262

WebSessionServerOAuth2AuthorizedClientRepository web session 411

WebTestClient class 98, **290**, **400**

Winkler, Scott 590

Wong, David 418

worker nodes 58, **217**

workflows 106

workflow scope 543, **546**

wrapper script 74

write:packages scope 186

Y

YAML

defining Deployment manifest with 224 – **227**

defining Service manifest with 234

Z

zero-trust 41

Previous chapter

< Appendix B Kubernetes in production with DigitalOcean